COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S3 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	8/4/26	**DATE SUBMITTED:**	8/4/26
NAME:	Busa Emmanuel A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

main.cpp ● main.exe C queueLL.h ●

ILOB > C queueLL.h > checkFront<T>(qNode<T>**)

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;

};

template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;

    newNode->data = newData;
    newNode->next = nullptr;


    return newNode;


}

template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* newNode = new_node(newData);

    if((*frontPtr)==nullptr && (*backPtr)==nullptr){

        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){

    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);


```

main.cpp ● main.exe queueLL.h ●

ILOB > queueLL.h > checkFront<T>(qNode<T>**)

```
1   #ifndef QUEUELL_H
26  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
37      (*backPtr) = newNode;
38  }
39
40  template <typename T>
41  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
42
43      qNode<T>* deleteNode = nullptr;
44      deleteNode = (*frontPtr);
45
46
47      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
48          (*frontPtr) = nullptr;
49          (*backPtr) = nullptr;
50          delete deleteNode;
51          return;
52
53      }
54
55      (*frontPtr) = (*frontPtr)->next;
56      deleteNode->next = nullptr;
57      delete deleteNode;
58
59  }
60
61  template <typename T>
62  T checkFront(qNode<T>** frontPtr){
63      return (*frontPtr)->data;
64
65
66  }
67
68  template <typename T>
69  T checkBack(qNode<T>** backPtr){
70      return (*backPtr)->data;
71  }
72
73
74  template <typename T>
75  void displayAll(qNode<T>** frontPtr){
76      qNode<T>* current = (*frontPtr);
77
78      while(current != nullptr){
79
```

main.cpp ● main.exe queueLL.h ●

ILOB > queueLL.h > checkFront<T>(qNode<T>**)

```
1   #ifndef QUEUELL_H
26  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
37      (*backPtr) = newNode;
38  }
39
40  template <typename T>
41  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
42
43      qNode<T>* deleteNode = nullptr;
44      deleteNode = (*frontPtr);
45
46
47      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
48          (*frontPtr) = nullptr;
49          (*backPtr) = nullptr;
50          delete deleteNode;
51          return;
52
53      }
54
55      (*frontPtr) = (*frontPtr)->next;
56      deleteNode->next = nullptr;
57      delete deleteNode;
58
59  }
60
61  template <typename T>
62  T checkFront(qNode<T>** frontPtr){
63      return (*frontPtr)->data;
64
65
66  }
67
68  template <typename T>
69  T checkBack(qNode<T>** backPtr){
70      return (*backPtr)->data;
71  }
72
73
74  template <typename T>
75  void displayAll(qNode<T>** frontPtr){
76      qNode<T>* current = (*frontPtr);
77
78      while(current != nullptr){
79
```

ILOB > main.cpp > ...

```cpp
#include<iostream>
#include "queueLL.h"


int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('L', &front, &back);
    std::cout<< front->data << " " << back->data << std::endl;
    enqueue('R', &front, &back);
    std::cout<< front->data << " " << back->data << std::endl;
    enqueue('S', &front, &back);
    std::cout<< front->data << " " << back->data << std::endl;


    dequeue(&front, &back);
    std::cout<< front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout<< front->data << " " << back->data << std::endl;

    std::cout<< "The front of the line is " << checkFront(&front) << std::endl;
    std::cout<< "The back of the line is " << checkBack(&back) << std::endl;
    std::cout<< "The queue contains: ";

    displayAll(&front);


    dequeue(&front, &back);

return 0;
}

```

main.cpp ● main.exe queueLL.h C/C++: g++.exe

ILOB > main.cpp > main()

```cpp
#include<iostream>
#include "queueLL.h"


int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('L', &front, &back);
    std::cout<< front->data << " " << back->data << std::endl;
    enqueue('R', &front, &back);
    std::cout<< front->data << " " << back->data << std::endl;
    enqueue('S', &front, &back);
    std::cout<< front->data << " " << back->data << std::endl;


    dequeue(&front, &back);
    std::cout<< front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout<< front->data << " " << back->data << std::endl;


    std::cout<< "The front of the line is " << checkFront(&front) << std::endl;
    std::cout<< "The back of the line is " << checkBack(&back) << std::endl;
    std::cout<< "The queue contains: ";

    displayAll(&front);


    dequeue(&front, &back);

return 0;
}
```

```
 E M M A N
Is the queue empty? 0
In the back of the queue is:  N
In the front of the queue is:  E
PS C:\Users\TIPQC\Documents\ILOB_BUSA>
```

```
L L
L R
L S
R S
S S
PS C:\Users\TIPQC\Documents\ILOB_BUSA>
```

Observation

I observed that in the code you need to always be aware of the element that is in the front or the back. It is kind of like the head and nodes when you code it that you need to always track where the addresses are.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.